

Aufgabe Funktionales Programmieren – Space Mission Control

Kontext & Lernziele

Bisher hast du Methoden immer fest in Klassen programmiert. In dieser Übung lernst du das **funktionale Programmierparadigma** in Java kennen. Du programmierst das Kontrollzentrum für eine Flotte von Raumschiffen.

Was du hier lernst:

1. **Funktionen als Werte:** Du lernst, dass man Funktionen genau wie Zahlen oder Strings in Variablen speichern und weitergeben kann.
2. **Lambda-Ausdrücke:** Die kompakte Schreibweise (parameter) -> { code }, um Funktionen in-line zu erstellen.
3. **Funktionale Interfaces:** Die Standard-Werkzeuge von Java (Predicate, Function, Consumer) und wie man eigene definiert.

1) Standard Functional Interfaces

In der Klasse MissionControl sollst du Verhalten in Variablen speichern. Nutze dafür Lambda-Ausdrücke.

1. **Ein Predicate<Spaceship> namens isLowOnFuel:**
 - Prüft ein Raumschiff. Es soll true zurückgeben, wenn das Tanklevel (fuelLevel) unter 20.0 liegt.
2. **Eine Function<Spaceship, String> namens generateStatusReport:**
 - Nimmt ein Raumschiff entgegen und gibt einen formatierten String zurück, z.B.: *"Raumschiff Apollo (Klasse: Explorer) - Schild: 85%"*.
3. **Ein Consumer<Spaceship> namens sendWarningSign:**
 - Nimmt ein Raumschiff entgegen und gibt eine Warnung in der Konsole (System.out.println) aus: *"WARNUNG: [Name] benötigt sofortige Wartung!"*

2) Ein eigenes Functional Interface

Manchmal reichen die Standard-Interfaces von Java nicht aus, besonders wenn wir mehr als zwei Parameter brauchen.

1. Erstelle ein eigenes Interface namens FleetCommander.
2. Verknüpfe es mit der Annotation @FunctionalInterface.

3. Definiere darin **genau eine** abstrakte Methode namens `assignMission`. Diese Methode soll drei Parameter akzeptieren:
 - Ein `Spaceship` (das Schiff)
 - Einen `String` (das Ziel, z. B. „Mars“ oder „Mond“)
 - Einen `int` (die benötigte Crew-Anzahl)
 - Die Methode soll einen `boolean` zurückgeben (ob die Mission gestartet werden kann).
4. **Implementierung:** Lege in der `MissionControl` eine Variable vom Typ `FleetCommander` an und implementiere sie mit einem Lambda-Ausdruck. Die Logik soll `true` zurückgeben, wenn die Crew-Anzahl größer als 2 ist und das Ziel nicht "Black Hole" lautet.

3) Funktionen als Parameter übergeben (High-Order Functions)

Das wahre Potenzial entfaltet sich, wenn wir Funktionen an andere Methoden übergeben.

- Schreibe in der `MissionControl`-Klasse eine Methode `public void processFleet(...)`.
- Diese Methode benötigt drei Parameter: Eine Liste von Raumschiffen, ein Predicate (als Filter) und einen Consumer (als Aktion).
- **Die Logik:** Iteriere durch die Liste der Raumschiffe. Wenn ein Raumschiff das Predicate erfüllt, führe den Consumer für dieses Raumschiff aus.
 - **Bonus:** implementiere diese Methode mit Hilfe von Streams.